

hermes-agent

A fully local agentic workflow system built on a Nous Research Hermes model served by Ollama. No cloud APIs, no telemetry, no network after the initial model pull.

Version 0.1.0

Runtime Python 3.11+ · Ollama

Default model hermes3:8b

Tests 271 passing

1. What this is

An agent loop, a tool system, and a workflow engine that run entirely on one machine. The agent loop is written by hand — no LangChain, no LlamaIndex — so every decision (when to stop, what gets sent, how a failure is reported back to the model) is readable in a single file.

The design assumption that shapes everything: **an 8B model running locally is capable but unreliable in specific, predictable ways**. Most of the engineering here is absorbing those failure modes rather than pretending they do not happen. Section 6 is the honest account of that.

2. Getting it running

STEP 1 — INSTALL OLLAMA AND START THE DAEMON

```
# https://ollama.com/download - or on Windows:
winget install Ollama.Ollama

ollama serve          # leave running in its own terminal
```

STEP 2 — PULL THE MODELS

```
ollama pull hermes3:8b      # the agent (~4.7 GB at Q4_K_M)
ollama pull nomic-embed-text # only for long-term memory (~274 MB)
```

STEP 3 — INSTALL THE PACKAGE

```
python -m pip install -e ".[dev]"      # add ,server for the HTTP API
cp .env.example .env                   # optional
```

STEP 4 — VERIFY THE WHOLE CHAIN BEFORE TRUSTING IT

```
hermes doctor
```

`doctor` checks that Ollama is reachable, that *both* model tags are actually pulled, and prints the resolved workspace, the tool list, and whether destructive tools are enabled. It exits non-zero on any problem. Run it first whenever something behaves oddly — it distinguishes "daemon down" from "model not pulled", which are the two failures that otherwise look identical.

EXPECTED OUTPUT WHEN OLLAMA IS NOT RUNNING

```
ollama unreachable - Cannot reach Ollama at http://localhost:11434. Start it with `ollama serve`
(or check HERMES_HOST).
```

Everyday commands

COMMAND	WHAT IT DOES
<code>hermes doctor</code>	Pre-flight: daemon, models, workspace, tools
<code>hermes chat</code>	Interactive REPL with tools and token streaming
<code>hermes chat "question"</code>	One-shot: send a message and exit
<code>hermes run <workflow></code>	Execute a declarative multi-step workflow
<code>hermes tools list --schemas</code>	Registered tools and their generated JSON schemas
<code>hermes workflows list / show <n></code>	Available workflows; step graph and branch conditions
<code>hermes trace</code>	List recent runs
<code>hermes trace <run_id></code>	Replay one run step by step
<code>hermes memory index <path></code>	Embed workspace docs into long-term memory
<code>hermes memory search <query></code>	Query long-term memory

USEFUL FLAGS

```
hermes chat --show-trace "... "          # print every tool call as it happens
hermes chat --tools calculator,read_file # restrict the tool set
hermes chat -m hermes3:70b --num-ctx 16384
hermes chat --temperature 0.0 --no-stream
hermes run doc_research -i folder=docs --allow-destructive
```

--ALLOW-DESTRUCTIVE

`write_file` and `run_python` refuse to run without it. A workflow that declares `allow_destructive: true` is rejected *before any model call is made*, so you get a config error rather than a half-finished run.

Configuration

Precedence, highest first: **CLI flags** → **HERMES_* env vars** → **.env** → **config.yaml** → **defaults**. Nested keys use a double underscore; a few flat aliases exist for convenience.

```
HERMES_MODEL=hermes3:70b          # alias for ollama.model
HERMES_OLLAMA__NUM_CTX=16384
HERMES_AGENT__MAX_ITERATIONS=15
HERMES_SAFETY__ALLOW_DESTRUCTIVE=true
```

Changing model is configuration only — no code changes, and larger Hermes variants work unmodified.

3. What it can do

The six built-in tools

TOOL	FLAGS	BEHAVIOUR
<code>read_file</code>		Read a UTF-8 file; sandboxed, byte-capped, truncation is labelled
<code>write_file</code>	destructive , serial	Write/append; creates parent directories; sandboxed
<code>list_dir</code>		List files, optionally recursive, entry-capped
<code>run_python</code>	destructive , serial	Subprocess, workspace cwd, no network, wall-clock timeout
<code>calculator</code>		Exact arithmetic via a whitelisted AST walker — never <code>eval</code>
<code>web_search</code>	stub	Interface only; pluggable backend, unset so the system stays offline

Capabilities

Tool-using chat

ReAct loop with streaming, per-turn concurrent tool execution, and hard guards on iterations, calls per turn, and repeated calls.

Declarative workflows

Multi-step YAML pipelines. Each step is one agent call with its own system prompt, tool subset, iteration budget and output schema. Supports sequential flow, conditional branching on structured output, and bounded loops.

Structured output

Ollama's `format` parameter constrains generation to a JSON schema; the result is validated and repaired once with the exact validation error fed back.

Short-term memory

Rolling context window that summarizes evicted turns before they fall off the end, so a long run does not silently lose its task.

Long-term memory (optional)

SQLite + `nomic-embed-text` embeddings, behind a flag. Retrieved chunks are injected as system context labelled untrusted.

Observability

One JSONL file per run, one record per step: model calls with token counts and latency, tool calls with arguments and results, fallback parses, loop detections, context trims, workflow transitions.

HTTP API (optional)

FastAPI with SSE streaming: `/health`, `/tools`, `/workflows`, `/chat`, `/chat/stream`, `/workflows/{name}/run`, `/runs/{id}`.

The two bundled workflows

DOC_RESEARCH — READ A FOLDER, WRITE A REPORT

```
hermes run doc_research -i folder=docs -i report_path=summary.md --allow-destructive
```

Five steps: inventory → read and note → synthesise → write report, with an early-exit branch when the folder holds no readable documents. Demonstrates conditional branching, a required output schema, and an optional schema that degrades to prose rather than failing the run.

CODE_FIX_LOOP — PLAN, WRITE, TEST, FIX

```
hermes run code_fix_loop \  
  -i task="parse 'HH:MM:SS' into seconds, raising ValueError on bad input" \  
  --allow-destructive
```

Six steps with a bounded repair loop: the `run_tests` step branches back to `fix` until the tests pass or `state.visits.run_tests` exceeds `inputs.max_fixes`. Critically, it gates on *actual test output*, not on the model's opinion of its own work.

Observability in practice

```
# Slowest tool calls across every run  
cat runs/*.jsonl | jq -r 'select(.kind=="tool_call") | [.latency_ms, .name] | @tsv' | sort -rn |  
head  
  
# How often did the fallback parser have to rescue a tool call?  
cat runs/*.jsonl | jq -r 'select(.kind=="parse_fallback") | .meta.recovered[]' | sort | uniq -c
```

That second number is the single most useful signal for judging whether a given model and quantization are reliable enough for your workload.

4. Architecture

Seven layers, each depending only on those below it. The arrows are imports, not network calls — everything runs in one process against a local HTTP daemon.

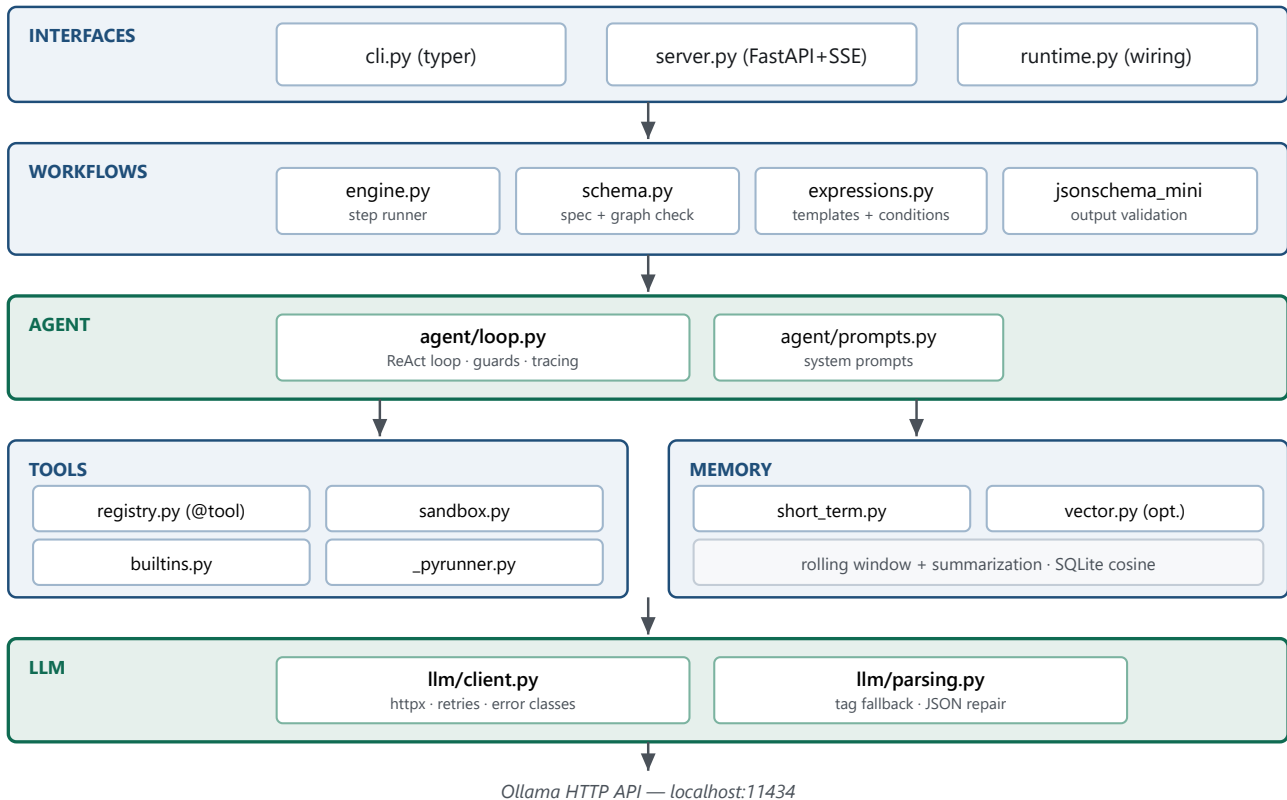


Figure 1. Layer map. `config.py`, `errors.py` and `trace.py` are cross-cutting and used by every layer.

The agent loop

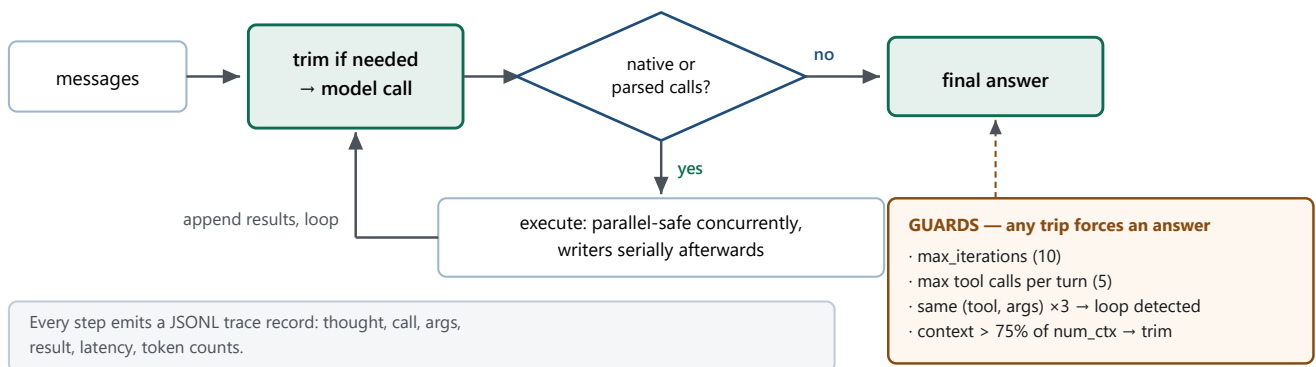


Figure 2. One iteration. When a guard trips, the loop makes one final call with *tools withheld* so the model cannot ask for more; if even that yields nothing usable, a degraded answer returns the last successful tool result with an honest explanation. The loop never returns an empty string.

Key design decisions

DECISION	RATIONALE
Raw <code>httpx</code> , not the <code>ollama</code> package	Control over retry policy, NDJSON stream handling, and error classification — "daemon down" vs. "model not pulled" must be distinguishable
Hand-written loop	Debuggability. With a local model you read the trace constantly; framework indirection makes that painful
Schemas derived from type hints	The schema cannot drift from the implementation
Declared, not inferred, parallelism	<code>parallel_safe=False</code> is explicit. Inferring a dependency graph from arguments is guesswork
No Jinja, no <code>eval</code>	Templates substitute; conditions walk a whitelisted AST. Model and tool output flows through both
SQLite over ChromaDB	Ships with Python, no native wheel, fast enough at single-machine scale. Swapping it means reimplementing two methods
One repair retry, not a loop	A model that fails twice with the error in front of it essentially never succeeds on the third try

5. Understanding Hermes

Who makes it

Nous Research is an open-weights AI collective. Hermes is their long-running series of instruction-tuned fine-tunes — they generally do not pretrain base models, they post-train other people's (Llama, Mistral, Qwen) with their own curated instruction and function-calling datasets.

The lineage

GENERATION	BASE	WHAT IT ADDED
Nous-Hermes	Llama / Llama 2	The original instruction tune
OpenHermes	Mistral 7B	Fully open dataset; strong small-model performance
Nous Hermes 2	Mistral / Mixtral / SOLAR	ChatML format, broader capability
Hermes 2 Pro	Mistral 7B, Llama-3 8B	The function-calling milestone. Introduced the <code><tools></code> / <code><tool_call></code> / <code><tool_response></code> convention and a dedicated JSON structured-output mode
Hermes 3 (Aug 2024)	Llama 3.1 — 8B, 70B, 405B	Agentic use, much stronger system-prompt adherence and steerability, long context, improved tool use and structured output. This project's default.

ON NEWER GENERATIONS

Nous has continued the series past Hermes 3, with later releases emphasising toggleable/hybrid reasoning. I cannot verify current tag names or exact base models from this offline environment — check ollama.com/library for what is actually pullable today. This system is tag-agnostic: any Hermes variant works by changing `ollama.model`, with no code changes.

Why Hermes specifically, for an agent

- **Function calling is a first-class training objective**, not an afterthought. Hermes 2 Pro onward were explicitly trained on tool-call datasets. Many comparable open models emit plausible-looking JSON that does not match the schema.
- **Strong system-prompt adherence.** Nous markets Hermes 3 as highly steerable with a deliberately thin "assistant persona". For an agent that is exactly what you want: the system prompt defines behaviour, and the model follows it rather than reverting to a trained-in personality.
- **Native structured output.** Pairs well with Ollama's `format` parameter for schema-constrained generation.
- **Open weights, permissive-ish licensing** (inherits the base model's licence — Llama 3.1 community licence for Hermes 3), and it runs offline.

The wire format — what actually goes over the wire

Hermes uses **ChatML** (`<|im_start|>` / `<|im_end|>`). Tool schemas are injected into the system turn inside a `<tools>` block, the model replies with a `<tool_call>` block, and results come back in `<tool_response>`:

```

<|im_start|>system
You are a function calling AI model. You are provided with function
signatures within <tools></tools> XML tags...
<tools>
{"type": "function", "function": {"name": "calculator", ...}}
</tools><|im_end|>
<|im_start|>user
What is 1234 * 5678?<|im_end|>
<|im_start|>assistant
<tool_call>
{"arguments": {"expression": "1234*5678"}, "name": "calculator"}
</tool_call><|im_end|>
<|im_start|>tool
<tool_response>
{"result": "7006652"}
</tool_response><|im_end|>

```

You normally never see this. Ollama's Hermes 3 model file template builds those blocks from the `tools` array you pass to `/api/chat`, and parses `<tool_call>` back out into `message.tool_calls`. That is the "native" path this system uses first.

WHY THE FALLBACK PARSER EXISTS

The native path works most of the time — not all of the time. An 8B model regularly emits the raw tags as plain *content*, truncates the closing tag mid-generation, writes a Python dict with single quotes, or leaves a trailing comma. When native parsing yields nothing, `llm/parsing.py` scans the text and performs bounded repairs. Note also that Hermes writes `{"arguments": ..., "name": ...}` — arguments first — so key order must not matter to your parser.

Hardware guidance

Pick the largest quantization that leaves roughly 2 GB of headroom. Ollama spills to CPU when a model does not fit in VRAM — it still works, just several times slower.

VRAM / RAM	TAG	NOTES
8 GB VRAM	<code>hermes3:8b (Q4_K_M)</code>	~4.7 GB. Comfortable at <code>num_ctx: 8192</code> . The default.
12 GB VRAM	<code>hermes3:8b-q8_0</code>	~8.5 GB. Noticeably better tool-call accuracy than Q4.
16 GB VRAM	<code>hermes3:8b-fp16</code>	Prefer this over a heavily quantized 70B — Q2 70B is usually <i>worse</i> at tool calls.
24 GB VRAM	<code>hermes3:70b-q4_K_M</code>	~40 GB total; expect CPU spill and roughly 3–8 tok/s.
CPU only, 16 GB	<code>hermes3:8b</code>	Works. Raise <code>ollama.request_timeout_s</code> to 600.
CPU only, 32 GB	<code>hermes3:8b-q8_0</code>	Better quality, still slow. Keep <code>num_ctx</code> at 8192.

Context length costs memory quadratically in KV cache. On 8 GB, `num_ctx: 8192` is roughly the ceiling for the 8B at Q4. If the model seems to forget early instructions, check the `context_trim` records in the trace before raising it.

6. Where small local models break — and the mitigations

This is the part that matters. Each risk below has specific, tested code behind it.

1 · Tool calls that are almost valid

Mitigation: a backstop parser handling tagged calls, fenced JSON and bare JSON, with bounded repairs — smart quotes, trailing commas, bracket balancing for truncated output, and Python-literal syntax.

```
parse_tool_calls('<tool_call>{"name": "calculator", "arguments": {"expression": "2+2"},}')  
# → ParsedCall(name='calculator', arguments={'expression': '2+2'}, repaired=True)
```

Every fallback parse is recorded as a `parse_fallback` step with a `repaired` flag, so you can *measure* how often your model needs it. The bare-JSON path is gated on known tool names — otherwise a structured JSON *answer* containing a `name` field gets misread as a tool call.

2 · Invalid arguments

Mitigation: validation failures never raise out of the loop. The error **plus the full schema** goes back to the model as an ordinary tool result, and Hermes usually self-corrects on the next turn. Unknown tool names get the same treatment, listing what is available.

```
ERROR (read_file): Invalid arguments for read_file: max_bytes: Input should be  
a valid integer... Expected schema: {...}. Call the tool again with corrected  
arguments.
```

3 · Infinite loops

Mitigation: three independent guards — a signature counter on `(tool, arguments)`, a per-turn call cap, and a global iteration cap — then a forced tool-free answer turn.

4 · JSON that does not match the schema

Mitigation: three layers. Ollama's `format` constrains generation; the result is validated; one repair retry carries the precise validation error and the schema. Steps may set `output.required: false` to degrade to prose instead of failing.

5 · Context overflow

Exceeding `num_ctx` makes Ollama silently drop the *start* of the prompt — usually the system prompt and the task.

Mitigation: usage is estimated at chars/4, then calibrated against the `prompt_eval_count` Ollama reports after every call. Past `context_trim_ratio`, evicted turns are summarized. Eviction operates on **blocks**: an assistant message with `tool_calls` and its `tool` replies move together, because splitting them produces a malformed prompt. The first user message is always preserved.

6 · Fabricated tool results

Models sometimes report a file as written without calling `write_file`.

Mitigation: partly the prompt, but mainly the trace. Every `tool_call` record holds the real arguments, result and latency. **If it is not in the JSONL, it did not happen.**

7. Safety model

- **Filesystem.** Every file tool routes through `resolve_in_workspace()`, which resolves to an absolute real path and requires it to sit under `WORKSPACE_DIR`. Resolving *before* comparing defeats `../` traversal, absolute paths, and symlinks pointing outside.
- **Destructive tools.** Gated behind an explicit flag; workflows declaring the need are refused up front.
- **Prompt injection.** Tool results are wrapped with an explicit instruction that they are untrusted data. The real boundary is the workspace and the destructive flag — not the prompt.

RUN_PYTHON IS NOT A SECURITY BOUNDARY

It runs under `python -I` with a scrubbed environment, a wall-clock timeout, `socket` neutered in-process (which blocks every stdlib HTTP client), and POSIX rlimits on memory, CPU, file size and process count. That is defence in depth against a *confused* model, not protection against hostile code. **Windows has no rlimit equivalent**, so only the network block and the timeout apply there, and `ctypes` can escape regardless. Run genuinely untrusted code in a container or VM.

8. Extending it

Adding a tool

Write a typed function with a Google-style docstring and decorate it. The JSON schema is derived from the signature, so it cannot drift from the code.

```
from hermes_agent.tools.registry import tool
from hermes_agent.tools.sandbox import resolve_in_workspace

@tool(tags=["fs"], destructive=False, parallel_safe=True)
def count_lines(path: str, skip_blank: bool = False) -> str:
    """Count the lines in a workspace text file.

    Args:
        path: Path relative to the workspace root.
        skip_blank: Ignore blank lines in the count.
    """
    text = resolve_in_workspace(path, must_exist=True).read_text(encoding="utf-8")
    lines = text.splitlines()
    if skip_blank:
        lines = [ln for ln in lines if ln.strip()]
    return f"{len(lines)} lines"
```

The contract: **parameter types** become the schema (prefer plain `str`, `int`, `float`, `bool`, `list[str]` — avoid `Any`). The **docstring summary** becomes the tool description and **Args:** entries become parameter descriptions; both go into the prompt, so write them for the model. **Raise on failure** — the registry converts it to a structured error. Async functions work; sync ones run in a thread. Verify with `hermes tools list --schemas`.

Writing a workflow

```
name: my_workflow
allow_destructive: false
max_steps: 12
inputs:
  topic: { required: true }
steps:
  - id: gather
    system: You are precise and cite filenames.
    prompt: Find everything about {{ inputs.topic }} in the workspace.
    tools: [list_dir, read_file]      # [] = none; omit = all
    max_iterations: 6
    output:
      schema:
        type: object
        properties:
          found: { type: boolean }
          required: [found]
    next:
      - when: "steps.gather.output.found == false"
        goto: end
      - goto: report
```

THE ONE THING THAT TRIPS PEOPLE UP

prompt: and system: are **templates** — {{ dotted.path }} substitution. when: is an **expression** — *no braces*. Write inputs.max_fixes, not {{ inputs.max_fixes }}.

CONTEXT PATH	MEANING
inputs.<name>	Declared workflow inputs
steps.<id>.answer	That step's prose answer
steps.<id>.output.<field>	Validated structured output
steps.<id>.stop_reason	How that step's agent loop ended
state.visits.<id>	Times a step ran — use this to bound loops
state.step_count	Total steps executed

First matching next transition wins; no match falls through in order; goto: end terminates. Missing paths resolve to None rather than raising, so a branch referencing a step that has not run evaluates falsy.

9. Honest limitations

- **Tool-call accuracy is the bottleneck**, not reasoning. Check grep parse_fallback runs/*.jsonl before blaming the loop.
- **Temperature above ~0.4 degrades tool calling sharply**. Default is 0.2. Raise it for prose, not for tool-using steps.
- **More than ~8 tools in one prompt** measurably hurts selection accuracy. Use per-step tools: subsets.
- **Multi-step plans drift** around 5–6 tool calls. Prefer several narrow workflow steps over one long agent run — this is the main reason the workflow layer exists.

- **Deeply nested output schemas still fail** even with `format`. Keep them shallow and flat.
- **Self-assessment is unreliable.** A critic step will approve broken work; `code_fix_loop` gates on real test output instead.
- **70B is better at tool use** but often too slow for interactive loops once it spills to CPU. An 8B at `q8_0` is usually the better trade on 12–16 GB.

Verification status. 271 unit tests pass and lint is clean. The parser, agent loop, sandbox, workflow engine and HTTP API were exercised directly; both bundled workflows were run end-to-end against a scripted model. The 12 `@pytest.mark.ollama` integration tests have *not* been run against a live model — Ollama was not running in the build environment. They skip themselves automatically; run `ollama serve`, then `pytest -m ollama -v` to close that gap.